关于某道题的优化

题意

给定一个长度为 n 的序列 a_i，求有多少个区间满足每个数出现次数均为偶数。

$n \le 3 \times 10^4, a_i \le 10^6$

思路

双指针扫描每个区间，拿个桶，$O(1)$ 更新状态，然后 $O(n^2)$，可以得到 40 的好成绩。

代码一

```
#include<bits/stdc++.h>
#define y1 y3647
#define INF 1000000000
#define LL long long
#define pii pair<int,int>
using namespace std;
template<typename _type>
inline void read(_type &x){
    x=0;int f=1;char ch=getchar();
    while(ch!=45&&(ch>'9'||ch<'0'))ch=getchar();
    if(ch==45){f=-1,ch=getchar();}
    while(ch<='9'&&ch>='0'){x=x*10+ch-48;ch=getchar();}x*=f;
}
template<typename _type1,typename _type2>void cmin(_type1 &a,const _type2 b)
{if(a>b)a=b;}
template<typename _type1,typename _type2>void cmax(_type1 &a,const _type2 b)
{if(a<b)a=b;}
const int N=30005;
int i,j,k,n,s,t,m,ans;
int a[N],b[N];
unsigned cnt[1024*1024];
signed main()
{
    freopen("gf.in","r",stdin);
    freopen("gf.out","w",stdout);
    //freopen(".in","w",stdout);
    read(n);
    for(i=1;i<=n;i++)read(a[i]),b[i]=a[i];
    int now=0;
    for(i=1;i<=n;i++){

        memset(cnt,0,sizeof(cnt));
        now=0;
        for(j=i;j<=n;j++){
            if(cnt[a[j]])now+=(cnt[a[j]]&1)?1:-1;
            cnt[a[j]]+=1;
            ans+=now==0;
        }
```

```
    }
    cout<<ans<<endl;
    return 0;
}
```

优化

1.高速缓存原理

计算机有个东西叫高速缓存，可以优化内存访问延迟。

一次独立的内存操作会读取一个 $64Byte$ 的 $cacheline$，从指令发出到从内存收到数据需要约 200 个 CPU 周期，而 CPU 会将一些常用的数据塞进 $cache$，也就是高速缓存中，加速读取，寄存器的访问速度是最快的，只需要不到 1 个时间周期，$L1$ 缓存需要约 3 个时间周期，$L2$ 为 10 个左右，$L3$ 为 20 个周期。

具体时间视计算机本身有差别，但基本的比例是这个。

代码一中，我们只关注瓶颈。

```
now+=-1+((cnt[a[j]]&1)<<1);
now+=!cnt[a[j]];
cnt[a[j]]+=1;
ans+=now==0;
```

其中，对于 `cnt` 的访问和对于 `a` 的访问，由于高速缓存并不大，所以塞不下 `cnt`，因此我们每个循环都需要等一个 `200` 时间周期的延迟，这是相当致命的，实际上等待的延迟并没有 `200` 时间周期，因为 CPU 将部分数据还是放进来 `cache`，但具体放哪些是由 `CPU` 决定的。

对于这个致命的延迟，我们可以将数据离散化，然后就可以得到一个 `75pts` 的代码。

```
#include<bits/stdc++.h>
#define y1 y3647
#define INF 1000000000
#define LL long long
#define pii pair<int,int>
using namespace std;
template<typename _type>
inline void read(_type &x){
    x=0;int f=1;char ch=getchar();
    while(ch!=45&&(ch>'9'||ch<'0'))ch=getchar();
    if(ch==45){f=-1,ch=getchar();}
    while(ch<='9'&&ch>='0'){x=x*10+ch-48;ch=getchar();}x*=f;
}
template<typename _type1,typename _type2>void cmin(_type1 &a,const _type2 b)
{if(a>b)a=b;}
template<typename _type1,typename _type2>void cmax(_type1 &a,const _type2 b)
{if(a<b)a=b;}
const int N=30005;
int i,j,k,n,s,t,m,ans;
int a[N],b[N];
unsigned cnt[1024*30];
signed main()
```

```
{
    freopen("gf.in","r",stdin);
    freopen("gf.out","w",stdout);
    //freopen(".in","w",stdout);
    read(n);
    for(i=1;i<=n;i++)read(a[i]),b[i]=a[i];
    sort(b+1,b+n+1);m=unique(b+1,b+n+1)-b-1;
    for(i=1;i<=n;i++)a[i]=lower_bound(b+1,b+m+1,a[i])-b;
    int now=0;
    for(i=1;i<=n;i++){

        memset(cnt,0,sizeof(cnt));
        now=0;
        for(j=i;j<=n;j++){
            if(cnt[a[j]])now+=cnt[a[j]]&1?1:-1;
            cnt[a[j]]+=1;
            ans+=now==0;
        }
    }
    cout<<ans<<endl;
    return 0;
}
```

2.流水线模式和延迟隐藏

现代计算机各类硬件的设计都采用了流水线模式，将每条汇编指令的执行都划分为了不同的流水线。

也就是说 **CPU 可以不间断的向内存发出访问指令，这些指令经过一定延迟后，内存会不间断的向CPU传数据，这个过程中，我们就只等待了一个内存延迟。**打个比方，这玩意就像烧水，水壶很多，灌水和倒水的时间都很短，所以正确的方式是全部烧上等，而不是烧一个等一个。

所以，如果我们能够连续的发出内存访问指令，那么内存延迟可以被有效隐藏，但注意到代码中，访问了内存后执行了一些计算，而内存访问是依赖于这些计算的，这产生了一个依赖，我们必须等待计算完成之后才能进行访问，所以无法有效的隐藏延迟。

向内存写入数据也是需要等待延迟的。

3.流水线和依赖分析

上一部分简单介绍了流水线模式，在我们的 CPU 中，也采用流水线的设计。

一条汇编指令的执行在 CPU 上大致分为五个部分，分别是：取指，访(寄)存，计算，内存操作，写回。

CPU 在这五个部分的设计上也采用了流水线，一条指令开始访存时，另一条指令的取指就开始了。

而如果下一条指令对前一条指令有**数据依赖**，那么 CPU 会通过转发操作消除这种依赖，但如果下一条指令的内容对上一条指令有依赖（比如说 if），那么 CPU 就不得不停止流水线，向流水线中插入气泡以等待。

这会极大降低 CPU 的利用率，因此，内循环中的 if 是相当不应该的。

4.流水线和分支预测

事实上，硬件的设计者注意到了这种依赖，而 CPU 会对这种依赖做出预测，预测基于程序计数器的的原理和一些其它统计数据，正确率约在 65% 左右，如果分支预测出现错误，那么会花费两个时间周期去消除这个错误，所以我们需要通过算术方式避免分支。

方式如下：

```
#include<bits/stdc++.h>
#define y1 y3647
#define INF 1000000000
#define LL long long
#define pii pair<int,int>
using namespace std;
template<typename _type>
inline void read(_type &x){
    x=0;int f=1;char ch=getchar();
    while(ch!=45&&(ch>'9'||ch<'0'))ch=getchar();
    if(ch==45){f=-1,ch=getchar();}
    while(ch<='9'&&ch>='0'){x=x*10+ch-48;ch=getchar();}x*=f;
}
template<typename _type1,typename _type2>void cmin(_type1 &a,const _type2 b)
{if(a>b)a=b;}
template<typename _type1,typename _type2>void cmax(_type1 &a,const _type2 b)
{if(a<b)a=b;}
const int N=30005;
int i,j,k,n,s,t,m,ans;
int a[N],b[N];
unsigned short cnt[1024*30];
signed main()
{
    freopen("gf.in","r",stdin);
    freopen("gf.out","w",stdout);
    //freopen(".in","w",stdout);
    read(n);
    for(i=1;i<=n;i++)read(a[i]),b[i]=a[i];
    sort(b+1,b+n+1);m=unique(b+1,b+n+1)-b-1;
    for(i=1;i<=n;i++)a[i]=lower_bound(b+1,b+m+1,a[i])-b;
    int now=0;
    for(i=1;i<=n;i++){

        memset(cnt,0,sizeof(cnt));
        now=0;
        for(j=i;j<=n;j++){
            now+=-1+((cnt[a[j]]&1)<<1);
            now+=!cnt[a[j]];
            cnt[a[j]]+=1;
            ans+=now==0;
        }
    }
    cout<<ans<<endl;
    return 0;
}
```

目前是最快代码。